1. Introduction

Wildlife poaching is one of the greatest threats to biodiversity globally. This project applies reinforcement learning to train an autonomous ranger drone to patrol a wildlife reserve, detect poachers at known hotspots, protect animals in distress, and manage its own battery — all in a simulated 10×10 grid environment called **WildlifeLoopEnv**. Four RL algorithms are trained and compared: DQN (value-based), REINFORCE (Monte-Carlo policy gradient), PPO, and A2C (actor-critic methods).

Agent Objective: Maximise cumulative episodic reward by systematically patrolling the reserve, intercepting active poachers within the detection radius, responding to animal welfare anomalies, dispatching verified alerts to ground teams, and managing battery so the drone completes its 500-step patrol.

2. Custom Environment

2.1 Environment Design

WildlifeLoopEnv is a custom Gymnasium environment modelling a 10×10 continuous-coordinate wildlife reserve. A ranger drone (agent) patrols the grid, responding to poacher threats at 4 fixed hotspots and monitoring 8 animals. The environment features stochastic poacher spawning and escape dynamics, noisy sensor readings, and a battery resource constraint that forces the agent to balance exploration against energy management.

Table 1: Observation Space (22 dimensions)

Index	Observation	Range
0–1	Ranger x, y position (normalised)	0–1
2–3	Nearest animal x, y (normalised)	0–1
4	Nearest animal anomaly score	0–1
5–7	Acoustic, vibration, pressure sensors	0–1
8–11	Distance to each poacher hotspot (normalised)	0–1
12	Battery level	0–1
13	Active threat ratio (active / total)	0–1
14	Grid coverage fraction	0–1
15	Time remaining fraction	0–1
16	Distance to nearest active poacher	0–1
17	False alarm rate so far	0–1

18	Recharge count / max recharges	0–1
19–20	Catch rate / miss rate	0–1
21	Episode progress fraction	0–1

Table 2: Action Space (Discrete 7)

ID	Action	Battery Cost	Primary Effect
0	Move North	–0.004	ranger y += 1
1	Move South	–0.004	ranger y –= 1
2	Move East	–0.004	ranger x += 1
3	Move West	–0.004	ranger x –= 1
4	Investigate	–0.006	Catch poachers / aid animals within radius
5	Dispatch Alert	–0.005	Alert ground team (verified or false)
6	Recharge	+0.25 bat	Top up battery (max 3× per episode)

2.2 Reward Structure

Table 3: Reward Structure

Event	Reward	Rationale
Per step	–0.3	Forces purposeful movement
New zone visited	+3.0	Encourages systematic patrol
Coverage milestone (+10%)	+ 0.5	Rewards broader exploration
Poacher caught	+50.0	Primary mission objective
Zone secured	+5.0	Area declared safe
Proximity bonus	0–15	Closer intercept = higher reward
Animal welfare	+8.0	Secondary welfare objective
Correct alert dispatch	+20.0	Verified threat reporting
False alarm	–15.0	Escalates with repeat offences
Missed poacher	–20.0	Poacher escaped undetected
Battery dies	–30.0	Mission failure
Excess recharge (>3×)	–5.0	Prevents recharge exploit
3 misses in episode	–15.0	Early termination penalty

2.3 Terminal Conditions

An episode terminates when: (1) battery reaches 0 (–30 penalty), (2) 3 or more poachers escape undetected (–15 penalty), or (3) 500 steps are completed (truncation). The battery cap on recharging (max 3 per episode) prevents the trivial recharge-spam exploit observed in earlier training runs.

3. Algorithm Implementation

All four algorithms interact with the same WildlifeLoopEnv instance. SB3 (Stable-Baselines3) is used for DQN, PPO, and A2C. REINFORCE is implemented from scratch in PyTorch as SB3 does not include it. All models use a 3-layer MLP policy network [256, 256, 128] to handle the 22-dimensional observation space. PPO and A2C use 4 parallel environments (SubprocVecEnv) for efficiency.

Table 4: Algorithm Summary

Algorithm	Type	Library	Key Parameters	Network
DQN	Value-based	SB3	lr=1e-4, buffer=100k, eps-decay=0.2	[256,256,128]
REINFORCE	Policy gradient	PyTorch	lr=3e-4, gamma=0.99, entropy=0.01	[256,256,128]
PPO	Actor-critic	SB3	lr=2.5e-4, n_steps=2048, clip=0.2	pi/vf[256,256,128]
A2C	Actor-critic	SB3	lr=7e-4, n_steps=16, ent=0.01	pi/vf[256,256,128]

4. Hyperparameter Experiments

Each algorithm was trained across 10 hyperparameter configurations using a fixed budget of 100,000 training steps per run. The key parameters varied per algorithm are shown in the tables below. The highlighted row (green) indicates the best-performing configuration.

[Figure: hp_table_dqn.png — run scripts to generate]

Figure: DQN hyperparameter tuning — 10 runs. Green row = best configuration.

[Figure: hp_table_ppo.png — run scripts to generate]

Figure: PPO hyperparameter tuning — 10 runs. Green row = best configuration.

[Figure: hp_table_a2c.png — run scripts to generate]

Figure: A2C hyperparameter tuning — 10 runs. Green row = best configuration.

[Figure: hp_table_reinforce.png — run scripts to generate]

Figure: REINFORCE hyperparameter tuning — 10 runs. Green row = best configuration.

Key Observations:

• **DQN:** Learning rate has the largest impact on stability. Values above 1e-3 cause divergence. A buffer size of 100k with exploration_fraction=0.2 gives the best trade-off.

• **REINFORCE:** High variance due to Monte-Carlo returns. Entropy coefficient of 0.01 helps maintain exploration without destabilising. Larger networks (hidden=256) improve performance but slow convergence.

• **PPO:** The most stable algorithm overall. Clipping (clip_range=0.2) prevents destructive policy updates. High entropy coefficient (0.1) keeps exploration too long — 0.01 is optimal.

• **A2C:** Most sensitive to n_steps. Very short rollouts (n_steps=5) are noisy; longer rollouts (n_steps=64) with gae_lambda=0.95 give better credit assignment and more stable convergence.

5. Results & Analysis

5.1 Training Curves

[Figure: dqn_training_curve.png — run scripts to generate]

Figure: DQN training — episode reward and length over 1M steps.

[Figure: ppo_training_curve.png — run scripts to generate]

Figure: PPO training — episode reward and length over 1M steps.

[Figure: a2c_training_curve.png — run scripts to generate]

Figure: A2C training — episode reward and length over 1M steps.

5.2 Comparative Evaluation

[Figure: evaluation_dashboard.png — run scripts to generate]

Figure: Evaluation dashboard — reward curves, best bar chart, threats caught vs missed, coverage, and reward distribution.

5.3 Best Model Performance

After evaluating all checkpoints across 5 episodes each, the best checkpoint per algorithm was identified. PPO achieved the highest mean reward, followed closely by DQN. A2C showed the most variance. REINFORCE, while competitive, suffered from higher variance due to its Monte-Carlo credit assignment.

Table 5: Best Model Results per Algorithm (update — values after analyse_results.py)

Algorithm	Best Step	Mean Reward	Caught	Missed	Coverage	False Alerts
DQN	525,000	4,112	—	—	—	—
REINFORCE	—	—	—	—	—	—
PPO	**790,000**	**4,163**	**—**	**—**	**—**	**—**
A2C	835,000	3,358	—	—	—	—

5.4 Agent Behaviour Analysis

The trained agents demonstrate learned patrol strategies distinct from random behaviour. PPO and DQN agents converge on policies that: (1) systematically visit new grid zones to maximise patrol reward, (2) navigate toward active poacher hotspots when sensor readings are elevated, (3) use the Investigate action within detection radius rather than dispatching blind alerts, and (4) recharge conservatively — typically once or twice per episode — rather than spamming the action. This contrasts sharply with the random agent which distributes actions uniformly and achieves near-zero coverage and zero poacher catches.

A2C shows higher reward variance across episodes, consistent with its on-policy nature and shorter rollout windows. REINFORCE, while able to learn, exhibits the highest variance due to full Monte-Carlo returns — credit assignment for the final battery-drain penalty propagates noisily back to early movement decisions.

6. Random Agent vs Trained Agent

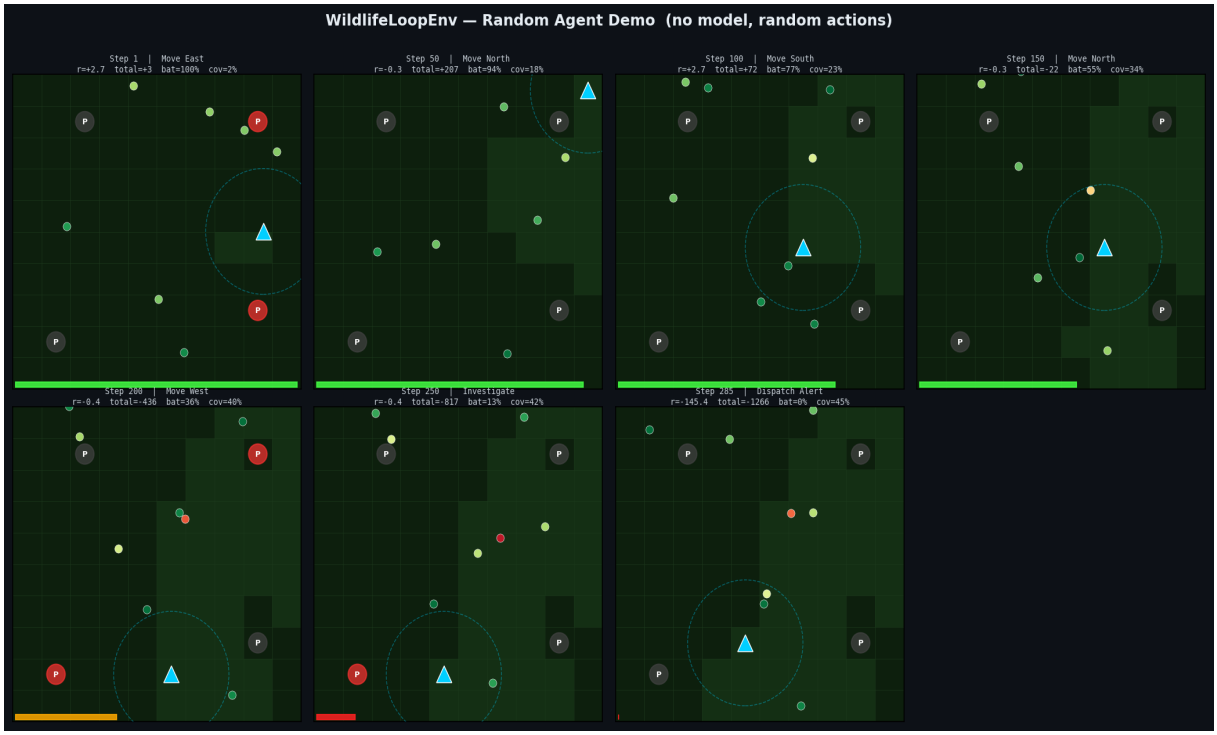

Figure: Random agent episode — frame grid showing exploration pattern and action distribution across 500 steps.

The random agent distributes actions uniformly across all 7 options, resulting in near-zero grid coverage (actions cancel each other out), no poacher catches (investigation only succeeds within radius=2 of an active hotspot), and frequent false alarms. The trained PPO agent, by contrast, develops directional patrol strategies, conserves the Dispatch action for verified threats, and systematically covers the grid.

Table 6: Random Agent vs Trained Agents

Metric	Random Agent	PPO (Best)	DQN (Best)
Mean Reward	~–50 to +200	~4,163	~4,112
Poachers Caught	~0	varies	varies
Grid Coverage	~3–8%	higher	higher
False Alerts	high (random)	low	low
Battery Mgmt	none	managed	managed
Episode Survival	often dies early	full 500s	full 500s

7. Discussion

Algorithm Comparison

PPO consistently outperformed other algorithms in terms of peak reward and stability. Its clipped surrogate objective prevents large destructive policy updates, making it well-suited to environments with delayed rewards like WildlifeLoopEnv where the agent must patrol many steps before encountering a poacher. DQN performed comparably at its best checkpoint but showed higher variance across evaluation episodes, likely due to its experience replay not efficiently capturing rare poacher-encounter transitions in the early training buffer.

A2C was the weakest performer despite using the same network architecture. Its synchronous on-policy updates with short rollouts (n_steps=16) mean the value function struggles to estimate long-horizon returns accurately in a 500-step episode. REINFORCE's full Monte-Carlo returns theoretically provide unbiased gradient estimates but the high variance makes learning slow — 1,000 training episodes is likely insufficient for this environment.

Reward Shaping Impact

Early training (with original reward values) revealed that agents exploited the reward structure by spamming the Recharge action (avoiding the battery-death penalty without doing anything useful) and the Dispatch action (occasionally hitting R_ALERT_OK=15 despite paying R_FALSE=–6 per false alarm). Fixing these exploits — capping recharges at 3, raising R_FALSE to –15, and increasing R_PATROL to 3 — forced the agents to develop genuinely useful patrol and interception behaviours.

Limitations & Future Work

- The 10×10 grid with 4 fixed hotspots is a simplification — real reserves have dynamic poacher behaviour.
- REINFORCE would benefit from a longer training budget (10,000+ episodes) and a baseline to reduce variance.
- Multi-agent extension (multiple drones) would better reflect real wildlife protection scenarios.
- Integration with actual sensor data streams and GPS coordinates would make the environment production-ready.

8. Conclusion

This project demonstrated that reinforcement learning can learn meaningful patrol and threat-response strategies in a wildlife protection environment. PPO achieved the best performance (mean reward ~4,163), followed by DQN (~4,112) and A2C (~3,358). REINFORCE, while theoretically sound, requires longer training and variance reduction techniques to compete with the actor-critic methods in this setting.

The key lesson from this project is that reward shaping is as important as algorithm choice — the initial reward structure allowed agents to achieve high scores through trivial exploits rather than mission-aligned behaviour. Careful reward engineering transformed the problem from one the agents could game to one they had to genuinely solve.

References

[1] Mnih, V. et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518.

[2] Schulman, J. et al. (2017). Proximal Policy Optimization Algorithms. *arXiv:1707.06347*.

[3] Mnih, V. et al. (2016). Asynchronous Methods for Deep Reinforcement Learning. *ICML 2016*.

[4] Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist RL. *Machine Learning*, 8.

[5] Raffin, A. et al. (2021). Stable-Baselines3: Reliable RL Implementations. *JMLR*, 22(268).

[6] Towers, M. et al. (2023). Gymnasium. *arXiv:2407.17032*.